

Lab 3

Instructions for Lab Submission:

1. File Naming and Submission:
 - a. Submit your solution as a zip file named **lab3_<ERP>.zip**, where *<ERP>* is your ERP number. For example, if your ERP number is *12345*, the file should be named *lab3_12345.zip*.
 - b. Ensure that the zip file contains:
 - i. C++ source files named as **task<number>.cpp** (e.g., **task1.cpp**).
 - ii. No unnecessary files (such as executables, temporary files, or editor configs).
 - c. Do not submit **TEXT(.txt)** files.
2. Code Organization:
 - a. Write each task in its own **.cpp** file (as specified above).
 - b. Ensure that your code is modular and logically separated into functions when specified.
3. Code Neatness:
 - a. Ensure your code is well-formatted and easy to read. Use proper indentation and meaningful variable names.
 - b. Include appropriate comments to explain the logic where necessary (especially for functions and complex sections of the code).
 - c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
4. Code Compilation and Functionality:
 - a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
 - b. Double-check that the functionality of the program meets the requirements outlined in the task.
5. Additional Instructions:
 - a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
 - b. Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
 - c. If you encounter any issues or require clarifications, reach out before the submission deadline.
6. Deadline:
 - a. Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted.
 - b. Double-check that the zip file is correctly named and contains all necessary files before submitting.
7. Plagiarism Policy:
 - a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

if-else Statements:

The if-else statement is one of the most basic tools in C++ for making decisions in code. It allows a program to perform different actions based on whether a condition is true or false. This kind of logic is called *branching*, and it's essential for writing programs that respond to user input, perform checks, or control flow.

A simple if statement looks like this:

```
if (x > 10) {  
    std::cout << "x is greater than 10\n";  
}
```

Here, the condition inside the parentheses is checked. If it's true, the code inside the braces is executed. If it's false, the code is skipped.

To provide an alternative path when the condition is false, we use else:

```
if (x > 10) {  
    std::cout << "x is greater than 10\n";  
} else {  
    std::cout << "x is 10 or less\n";  
}
```

We can also check multiple conditions using else if:

```
if (x > 10) {  
    std::cout << "x is greater than 10\n";  
} else if (x == 10) {  
    std::cout << "x is exactly 10\n";  
} else {  
    std::cout << "x is less than 10\n";  
}
```

The conditions are checked from top to bottom. As soon as one is true, its block runs and the rest are skipped. If none are true, the final else block (if present) runs.

Note: Conditions must always be inside parentheses, and the code blocks are usually wrapped in curly braces {}. If the block contains only one line, the braces are optional—but using them consistently helps avoid bugs.

Bit Shifting

Bit shifting is a low-level operation that directly manipulates the bits (0s and 1s) of an integer. It is often used for performance optimizations, dealing with binary data, or working with hardware.

There are two basic bit shift operators in C++:

- Left shift (<<)
- Right shift (>>)

Left Shift (<<): This operator shifts all bits to the left by a specified number of positions. For each shift left by **k** positions, the number is multiplied by 2^k .

Mathematically:

$x \ll k$ is equivalent to $x * 2^k$

Example:

```
int x = 3;           // Binary: 00000011
int y = x << 2;       // y = 3 * 2^2 = 12 (Binary: 00001100)
```

All bits in **x** are moved two positions to the left. Zeros are inserted from the right.

Right Shift (>>): This operator shifts all bits to the right. For each shift right by **k** positions, the number is divided by 2^k , discarding any remainder (i.e., integer division).

Mathematically:

$x \gg k$ is equivalent to $x / 2^k$

Example:

```
int x = 20;          // Binary: 00010100
int y = x >> 2;       // y = 20 ÷ 4 = 5 (Binary: 00000101)
```

Bits are moved two places to the right. For unsigned integers, zeros are inserted from the left.

Important Notes:

- Bit shifting only works with integers — not floats or doubles.
- Shifting by a negative number or beyond the size of the data type (e.g., more than 31 bits on a 32-bit int) leads to undefined behavior.
- Right shifting signed negative numbers can give implementation-defined results, since it depends on how the system handles sign bits (arithmetic vs. logical shifting).

Why Use Bit Shifting?

Bit shifts are extremely fast and efficient. In many cases, they replace multiplication or division when working with powers of 2:

```
int result = x * 8;      // Equivalent to: x << 3
int result = x / 4;      // Equivalent to: x >> 2
```

This can lead to faster code, especially in performance-critical situations like graphics processing, game development, or embedded systems.

Lab Tasks

Task 1:

Write a program that asks for a student's score (0-100) and prints their grade based on the score:

- 90-100: A
- 80-89: B
- 70-79: C
- 60-69: D
- Below 60: F

```
fall25/Lab_Solutions/lab03 on ʘ main
> g++ task1.cpp && ./a.out
Enter the student's score (0-100): 80
The student's grade is: B

fall25/Lab_Solutions/lab03 on ʘ main took 4s
> g++ task1.cpp && ./a.out
Enter the student's score (0-100): -1
Invalid score. Please enter a score between 0 and 100.

fall25/Lab_Solutions/lab03 on ʘ main
>
```

Task 2:

You are developing firmware for an embedded system that controls a series of status lights. Each light corresponds to a specific system component. The status of all 32 components is encoded in a single **unsigned int** variable, where each bit represents the status of one component. A 1 indicates the component is active (light on), and a 0 indicates it is inactive (light off). Your task is to write a program that can check if a specific status light is on.

Write a program that takes two inputs from the user:

1. An **unsigned int** representing the status word.
2. An integer **n** (0-31) representing the bit position (the component number).

The program should use a bitwise shift and an **if-else** statement to determine if the *n*th bit is set. The output should be a clear message indicating whether the status light is **ON** or **OFF**.

```
fall25/Lab_Solutions/lab03 on ʘ main
> ./a.out
Enter the status word (unsigned int): 12345
Enter the bit position (0-31): 2
The status light for component 2 is OFF.

fall25/Lab_Solutions/lab03 on ʘ main took 4s
> ./a.out
Enter the status word (unsigned int): 12345
Enter the bit position (0-31): 32
Error: Bit position must be between 0 and 31.
```

*Note: You **must** also handle edge cases.*

Task 3:

You are programming a micro-controller for a smart sensor. This sensor's raw data output is an **unsigned int**. Depending on the environmental conditions, the data needs to be adjusted. If the raw data is within a specific range, you need to perform a right bit shift to normalize the value. Otherwise, you need to perform a left bit shift to amplify the signal for further processing. This is a common technique for managing data precision and scale in embedded systems.

Write a C++ program that takes two inputs from the user:

1. An **unsigned int** representing the raw sensor data.
2. An integer **shift_amount** representing the number of positions to shift.

The program should use **if-else** statements to check the following conditions:

- If the raw data is between 100 and 500 (inclusive), perform a right bit shift by **shift_amount** and print the new value.
- Otherwise, perform a left bit shift by **shift_amount** and print the new value.

```
fall25/Lab_Solutions/lab03 on ʘ main
> g++ task2.cpp && ./a.out
Enter the raw sensor data (unsigned int): 50
Enter the shift amount (integer): 2
The adjusted sensor data after left shift is: 200
```